TravelMate Interview Cheat Sheet

30-second explanation

TravelMate is a Django MVT trip-planning website. Users can search destinations, hotels, transport options, and holiday packages, create accounts, submit booking requests, and view their private booking and browsing history. Django handles routing, ORM/database access, forms, authentication, sessions, CSRF, templates, migrations, and admin. The frontend is server-rendered HTML enhanced with CSS and plain JavaScript. It is not currently a REST API project.

Architecture

```
Browser
  -> tripplanner/urls.py
  -> accounts/urls.py or travel/urls.py
  -> function-based view
  -> Django form + ORM query
  -> SQLite database
  -> template rendered as HTML
  -> JavaScript enhances the page
```

Core models

- `Destination`: place, country, description, season.
- `Hotel`: belongs to a destination; price per night and amenities.
- `TransportOption`: belongs to a destination; plane/train/bus/cab, times, price, seats.
- `HolidayPackage`: belongs to a destination; duration, price, itinerary.
- `TripBooking`: user booking request with optional hotel, transport, or package.
- `BrowsingHistory`: authenticated user's viewed items.
- Django `User`: authentication and ownership of bookings/history.

Main flow to explain

1. URLconf maps the request to a view.
2. The view reads GET parameters or POST form data.
3. Django form validation checks the input.
4. The view queries models through the ORM.
5. `select_related()` loads foreign-key data efficiently.
6. The view sends a context dictionary to a template.
7. The template returns HTML.
8. JavaScript handles UI behavior such as filters, menus, conditional fields, and localStorage saved trips.

Booking flow

1. `/book/` is protected by `@login_required`.

2. GET displays `TripBookingForm` with the user's name and email prefilled.
3. POST binds submitted data to the `ModelForm`.
4. `form.is_valid()` runs field/model validation.
5. `form.save(commit=False)` creates the object without saving.
6. The view assigns `booking.user = request.user`.
7. `booking.save()` writes to SQLite.
8. Django messages shows confirmation.
9. Redirect to the success page uses Post/Redirect/Get.

Mention `{% csrf_token %}` for CSRF protection and `request.user` for ownership.

Important Django concepts

- MVT: models, views, templates.
- URL routing: `path()`, `include()`, named URLs, URL reversing.
- ORM: `filter()`, `Q()`, `order_by()`, `aggregate()`, `select_related()`.
- Forms: regular `Form` for search, `ModelForm` for booking.
- Auth: `User`, `login()`, `logout()`, sessions, `@login_required`.
- Middleware: sessions, authentication, CSRF, messages, security.
- Templates: inheritance with `{% extends %}`, blocks, context, filters.
- Migrations: versioned database schema changes.
- Admin: staff CRUD/search/filter interface.
- Management command: `seed_demo_data` loads repeatable demo inventory.

REST answer

The project has HTTP page endpoints, but no REST API. There is no Django REST Framework, serializer, API view, router, or JSON endpoint.

Say:

> This is currently a server-rendered Django MVT application rather than a REST API. To support a mobile app or separate frontend, I would add DRF serializers, API views or viewsets, `/api/` routes, authentication, permissions, pagination, filtering, and JSON responses.

Live demo order

1. Run the server.
2. Search by destination and budget.
3. Open a detail page and explain browsing history.
4. Sign up or log in.
5. Open the protected booking form.
6. Change booking type and show JavaScript field behavior.
7. Submit a booking.
8. Open My History and explain user filtering.
9. Open Django admin.
10. Explain that saved trips use browser localStorage, while bookings/history use SQLite.

Honest limitations

- Booking is a request, not payment or a supplier reservation.
- Inventory is demo data; no live travel-provider integration.
- The booking form needs stronger cross-field validation.
- Rooms/seats are not decremented transactionally.
- Development settings still use `DEBUG = True`, a placeholder secret key, and empty `ALLOWED_HOSTS`.
- No automated tests are currently present.
- No REST API is currently implemented.

Setup commands

```
python -m venv .venv
.\.venv\Scripts\activate
pip install -r requirements.txt
python manage.py check
python manage.py migrate
python manage.py seed_demo_data
python manage.py runserver
```

Open `http://127.0.0.1:8000/`.